

AWS Python Deployment

Session 1 — Server-Based Deployment

AmaliTech · Python Chapter

2025

Contents

Before You Start

Demo 1 — Single EC2 Instance Running Flask

Demo 2 — Load Balancer with Auto Scaling Group

Demo 3 — Automated Deployment with AWS CodeDeploy

Cleanup — Delete everything we created

Section A — CI/CD with GitHub Actions (optional, ~20 minutes)

Before You Start

What you need on your laptop

- An AWS account where you can create resources (the AWS Free Tier is enough)
- A web browser to use the AWS Management Console
- A terminal (this guide assumes **Ubuntu Linux** — every shell command works as-is)
- Python 3.12 installed for local testing
- A free GitHub account

What we will deploy

A small Python Flask application with three endpoints:

- `GET /` — returns a greeting
- `POST /echo` — echoes back JSON we send
- `GET /health` — returns `ok`, used by the load balancer

We use the **AWS Management Console** for every step in these demos. The Console is the visual interface at <https://console.aws.amazon.com>. We use it because:

- It is the easiest way to [see](#) what each AWS service looks like
- Every click in the Console maps to one API call — once you know the Console, the AWS CLI and Terraform make sense
- Mistakes are obvious and easy to undo

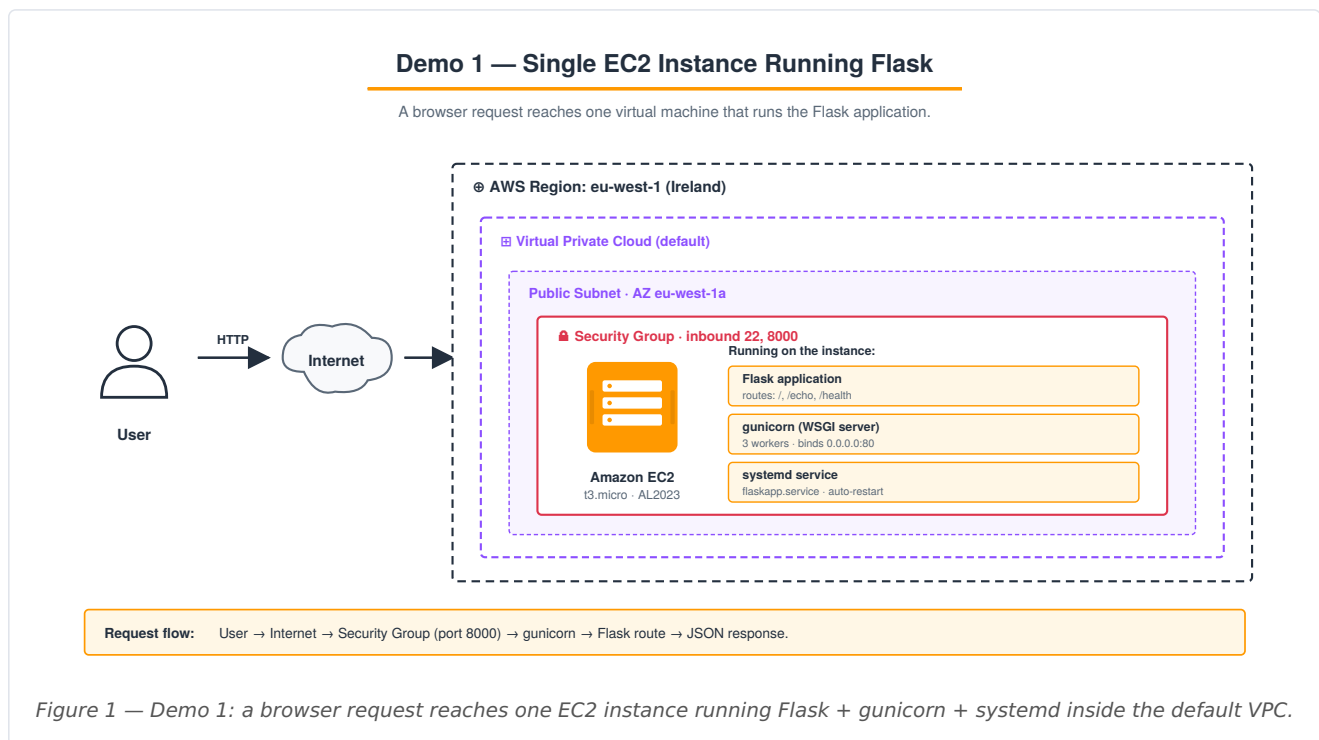
The Region we use

All three demos use `eu-west-1` (Ireland). It is the closest AWS Region to Ghana with full service availability. Set the Region in the top-right corner of the Console before doing anything.

Tip: put every resource you create today under a project tag so cleanup at the end is easy. We use `Project = python-chapter-demo`. The Console has a "Tags" section on every "Create..." form.

Demo 1 — Single EC2 Instance Running Flask

Goal: run our Flask app on one Linux virtual machine in AWS, reachable from any browser.



The project structure

We build this on our laptop first, test it locally, then push to GitHub and deploy.

```
flask-app/
├─ app.py           # The Flask application
├─ requirements.txt # Python dependencies
├─ templates/
│   └─ index.html  # The styled landing page
└─ README.md       # Notes for your team
```

Note: Flask automatically looks for HTML templates in a folder named `templates/` next to `app.py`. Keeping the page in its own file makes the Python code clean and the HTML easy to edit.

Step 1.1 — Create the local project and the Flask app

Open a terminal on your laptop and run:

```
# Create the project folder and the templates subfolder
mkdir -p flask-app/templates
cd flask-app
```

Now create the application file. Open your editor (VS Code, nano, vim, etc.) and create `app.py` with this content:

File: `app.py`

```
import socket
from flask import Flask, jsonify, request, render_template

app = Flask(__name__)

APP_VERSION = "v1"

@app.route("/")
def index():
    return render_template(
        "index.html",
        message="Hello from Flask on AWS!",
        version=APP_VERSION,
        host=socket.gethostname(),
    )

@app.route("/echo", methods=["POST"])
def echo():
    # Echo back whatever JSON was sent
    data = request.get_json(silent=True) or {}
    return jsonify({"you_sent": data, "version": APP_VERSION})

@app.route("/health")
def health():
    # Used by the load balancer in Demo 2
    return "ok", 200

if __name__ == "__main__":
    # Bind to 0.0.0.0 so it is reachable from outside the machine
    app.run(host="0.0.0.0", port=8000)
```

Now create the landing-page template. The `{{ version }}`, `{{ message }}` and `{{ host }}` placeholders are filled in by `render_template` above.

File: `templates/index.html`

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>Flask on AWS</title>
    <style>
      body {
        font-family:
          -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
        margin: 0;
        background: linear-gradient(135deg, #232f3e 0%, #1a2536 100%);
        color: #f5f6f8;
        min-height: 100vh;
        display: flex;
        align-items: center;
        justify-content: center;
      }
      .card {
        background: #ffffff;
```

```

        color: #1a1a1a;
        padding: 2.5rem 3rem;
        border-radius: 10px;
        box-shadow: 0 10px 40px rgba(0, 0, 0, 0.3);
        max-width: 520px;
        width: 90%;
    }
    h1 {
        margin: 0 0 1rem 0;
        color: #232f3e;
        border-bottom: 3px solid #ff9900;
        padding-bottom: 0.6rem;
    }
    .row {
        margin: 0.6rem 0;
    }
    .label {
        color: #5a6877;
        font-size: 0.9rem;
    }
    .value {
        font-weight: 600;
        font-family: monospace;
    }
    .badge {
        display: inline-block;
        background: #ff9900;
        color: #fff;
        padding: 2px 10px;
        border-radius: 12px;
        font-size: 0.85rem;
        font-weight: 700;
    }
    a {
        color: #ff9900;
    }
</style>
</head>
<body>
    <div class="card">
        <h1>Flask on AWS <span class="badge">{{ version }}</span></h1>
        <p>{{ message }}</p>
        <div class="row">
            <span class="label">Served by:</span>
            <span class="value">{{ host }}</span>
        </div>
        <div class="row">
            <span class="label">Try also:</span>
            <a href="/health">/health</a> &middot;
            <code>POST /echo</code>
        </div>
    </div>
</body>
</html>

```

Create the dependencies file:

File: requirements.txt

```

flask==3.0.3
unicorn==23.0.0

```

Why gunicorn? Flask's built-in server (`app.run`) is only for development — it is single-threaded and warns you not to use it in production. **gunicorn** is a production-grade WSGI server that runs multiple Python worker processes so the app can handle many requests at once.

Step 1.2 — Test the app locally before deploying

Always run the app on your laptop first. If it does not work locally, it will not work on AWS.

```
# If Python or venv is missing, install them first
sudo apt update
sudo apt install -y python3 python3-venv python3-pip

# Create a Python virtual environment
python3 -m venv venv

# Activate it
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Run the app
python app.py
```

You should see:

```
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8000
```

Open a **second terminal** (keep the app running in the first) and test:

```
# Test the root endpoint (returns HTML)
curl http://localhost:8000/

# Test the echo endpoint (returns JSON)
curl -X POST http://localhost:8000/echo \
  -H "Content-Type: application/json" \
  -d '{"name": "Ghana"}'

# Test the health endpoint
curl http://localhost:8000/health
```

For the best view, open `http://localhost:8000/` in your **browser** — you should see a styled card with the version badge, the message, and the hostname.

Stop the app with `Ctrl+C` in the first terminal. Local test passed — we can deploy.

Step 1.3 — Push the code to GitHub

CodeDeploy in Demo 3 will pull from a release zip, but for Demo 1 we **clone the code directly from GitHub onto the EC2 instance**. So push it now.

```
# Add a .gitignore so we do not commit the venv
echo "venv/" > .gitignore
echo "__pycache__/" >> .gitignore

# Initialise git
git init
git add .
git commit -m "Initial Flask app"

# Create a new public repo on GitHub called "flask-app"
# (use https://github.com/new in your browser)
# Then point your local repo to it:
git branch -M main
git remote add origin https://github.com/<your-username>/flask-app.git
git push -u origin main
```

Replace `<your-username>` with your GitHub username. The repo must be **public** so the EC2 instance can clone it without authentication.

Step 1.4 — Create an SSH key pair in the AWS Console

We need a key pair to SSH into the EC2 instance later.

In the AWS Console:

1. Make sure the Region in the top-right is **eu-west-1**
2. Search for **EC2** in the top search bar and open it
3. In the left sidebar under **Network & Security**, click **Key Pairs**
4. Click **Create key pair** (top right)
5. Name: `flask-demo-key`
6. Key pair type: **RSA**
7. Private key file format: **.pem**
8. Click **Create key pair**

Your browser will download `flask-demo-key.pem`. Move it to a safe location and lock down its permissions:

```
mkdir -p ~/.ssh
mv ~/Downloads/flask-demo-key.pem ~/.ssh/
chmod 400 ~/.ssh/flask-demo-key.pem
```

Why `chmod 400` ? SSH refuses to use a key file that is readable by anyone else on the machine. `400` means only you can read it.

Step 1.5 — Create a Security Group

A **Security Group** is a virtual firewall around the EC2 instance. It controls which ports are reachable and from where.

In the AWS Console:

1. Still in **EC2**, in the left sidebar under **Network & Security**, click **Security Groups**
2. Click **Create security group** (top right)
3. Name: `flask-demo-sg`
4. Description: `Security group for the Flask demo`
5. VPC: leave the default

Under **Inbound rules**, click **Add rule** twice and configure:

Type	Protocol	Port range	Source	Description
SSH	TCP	22	My IP	SSH from my laptop
Custom TCP	TCP	8000	Anywhere-IPv4 (0.0.0.0/0)	App from anywhere

Under **Outbound rules** leave the default (all traffic allowed out).

Scroll down, in **Tags**, click **Add new tag**: Key `Project`, Value `python-chapter-demo`.

Click **Create security group**.

Why two rules? Port 22 is SSH — we only allow it from our IP so randoms on the internet cannot try to log in. Port 8000 is where our Flask app listens — anyone in the world should be able to view it.

Step 1.6 — Launch an EC2 instance

Now we create the virtual machine itself.

In the AWS Console:

1. Still in **EC2**, in the left sidebar click **Instances**
2. Click **Launch instances** (orange button, top right)
3. **Name and tags**: `flask-demo-instance`. Click **Add additional tags** -> Key `Project`, Value `python-chapter-demo`.
4. **Application and OS Images (AMI)**: in the **Quick Start** tab select **Ubuntu**, then in the AMI dropdown choose **Ubuntu Server 22.04 LTS (HVM), SSD Volume Type** — make sure it says `64-bit (x86)`
5. **Instance type**: `t3.micro` (Free Tier eligible)
6. **Key pair (login)**: select `flask-demo-key`
7. **Network settings**: click **Edit** on the right.
 - Auto-assign public IP: **Enable**
 - Firewall: select **Select existing security group**
 - Common security groups: select `flask-demo-sg`
8. **Configure storage**: leave at 8 GB gp3
9. **Advanced details**: scroll down to **User data** and paste the script below

Then click **Launch instance** (bottom right).

This is the **user data** script — paste it into the "User data" text area in step 9:

```
#!/bin/bash
# EC2 user-data: runs once on first boot, as root.

# Exit on error, treat unset vars as errors, and print each command before running it.
set -eux

# Redirect all output to a log file for post-boot debugging via SSH.
exec > /var/log/user-data.log 2>&1

# Install system dependencies: Python, venv support, pip, and git.
apt update -y
apt install -y python3 python3-venv python3-pip git

# Clone the app into the ubuntu user's home (skip if already present, so re-runs are safe).
cd /home/ubuntu
[ -d flask-app ] || git clone https://github.com/<your-username>/flask-app.git
chown -R ubuntu:ubuntu flask-app

# Create an isolated virtualenv and install Python dependencies into it.
cd flask-app
[ -d venv ] || python3 -m venv venv
./venv/bin/pip install -r requirements.txt

# Write a systemd unit so gunicorn starts on boot and restarts on crash.
cat > /etc/systemd/system/flaskapp.service <<'EOF'
[Unit]
Description=Flask App
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/flask-app
ExecStart=/home/ubuntu/flask-app/venv/bin/gunicorn --workers 3 --bind 0.0.0.0:8000 app:app
Restart=always

[Install]
WantedBy=multi-user.target
EOF

# Load the new unit and start it now (also enables it for future boots).
systemctl daemon-reload
systemctl enable --now flaskapp
```

Replace `<your-username>` with your actual GitHub username before pasting.

What is User Data? It is a shell script that AWS runs the first time the instance boots. We use it to install Python, clone the code, install dependencies, and start the app — all without touching the machine manually.

Why systemd? It makes our app a real Linux service. If the app crashes, systemd restarts it. If the machine reboots, the app comes back up automatically. This is the production pattern on Linux.

Step 1.7 — Wait, then test the app

Back on the **Instances** page, click your instance and wait until the **Status check** column shows `2/2 checks`

passed (about 2-3 minutes). Then:

1. Copy the **Public IPv4 address** of the instance (shown in the Details panel)
2. Open `http://<public-ip>:8000/` in your browser — for example `http://52.18.75.57:8000/`

You should see the styled card showing **"Hello from Flask on AWS!"** along with the version badge and the EC2 hostname.

Important: the URL must include `:8000` — our app listens on port 8000, not port 80. If you forget the port, the browser will hang or show "site can't be reached".

If you see the styled page — your Flask app is live on AWS. Take a screenshot for the demo recap!

Test the other endpoints from your laptop:

```
PUBLIC_IP=<your-instance-public-ip>

curl http://$PUBLIC_IP:8000/health

curl -X POST http://$PUBLIC_IP:8000/echo \
  -H "Content-Type: application/json" \
  -d '{"name": "AmaliTech"}'
```

If the page does not load: wait another minute. The user-data script takes 60–90 seconds to install Python, clone the repo, and start gunicorn. If it still does not load after 3 minutes, SSH in and check (see next step).

Step 1.8 — SSH in to inspect (optional)

```
ssh -i ~/.ssh/flask-demo-key.pem ubuntu@<public-ip>

# Once connected:
sudo systemctl status flaskapp      # Is the service running?
sudo journalctl -u flaskapp -n 50   # Last 50 log lines
cat /var/log/user-data.log          # Output of the user data script
```

Type `exit` to leave the SSH session.

Step 1.9 — Alternative: install the app manually (no user-data)

User data is the cleanest path because the app is ready the moment the instance boots — no manual steps, no SSH. But sometimes you want to do it by hand: when you are **learning** and want to see each step, when you are **troubleshooting** a script that is not working, or when you are **preparing a custom AMI** you will use later.

If you want to do Demo 1 this way instead, the only change to Step 1.6 is: **leave the "User data" field empty**. Everything else (AMI, instance type, key pair, security group) stays the same. Then launch the instance.

Once the instance reaches **2/2 checks passed**, SSH in and run the commands below.

Important: every line below is run **on the EC2 instance** after you SSH in, not on your laptop.

```
# From your laptop, SSH into the instance
ssh -i ~/.ssh/flask-demo-key.pem ubuntu@<public-ip>
```

Once connected, you are now on the EC2 instance. Run:

```
# 1. Install system dependencies
sudo apt update -y
sudo apt install -y python3 python3-venv python3-pip git

# 2. Clone your application repo
cd /home/ubuntu
git clone https://github.com/<your-username>/flask-app.git
cd flask-app

# 3. Create a virtual environment and install Python dependencies
python3 -m venv venv
./venv/bin/pip install -r requirements.txt
```

At this point the app is installed but **not running yet**. You have two choices:

Option A — Quick foreground run (good for first-time verification):

```
./venv/bin/gunicorn --workers 3 --bind 0.0.0.0:8000 app:app
```

Open `http://<public-ip>:8000/` in your browser — you should see the styled card. Press `Ctrl+C` to stop the app. The problem with this approach: when you close the SSH session, the app stops.

Option B — Run as a systemd service (recommended, matches the user-data behavior):

This creates the same `flaskapp.service` the user-data script would have created, so the app starts on boot and auto-restarts on crash.

```
sudo tee /etc/systemd/system/flaskapp.service > /dev/null <<'EOF'
[Unit]
Description=Flask App
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/flask-app
ExecStart=/home/ubuntu/flask-app/venv/bin/gunicorn --workers 3 --bind 0.0.0.0:8000 app:app
Restart=always

[Install]
WantedBy=multi-user.target
EOF

# Load the new unit and start it now (also enables it for future boots)
sudo systemctl daemon-reload
sudo systemctl enable --now flaskapp

# Verify it is running
sudo systemctl status flaskapp
```

The last command should show `Active: active (running)` in green. Type `q` to exit the status view.

What `sudo tee ... > /dev/null` **does:** `tee` lets us write to a file that requires root (the `/etc/systemd/` directory needs sudo). `> /dev/null` discards `tee`'s stdout so the heredoc content is not echoed back to your terminal.

Test from your laptop:

```
curl http://<public-ip>:8000/health
# Expected: ok

curl http://<public-ip>:8000/
# Expected: HTML response
```

Or open `http://<public-ip>:8000/` in your browser — same styled card as before.

When to prefer user-data vs manual setup

- **User-data:** production, Auto Scaling Groups (Demo 2 needs this — new instances must self-install), and any time you want repeatability.
- **Manual setup:** learning, debugging a broken user-data script, building a "golden" AMI, or one-off experimentation.

What we built in Demo 1

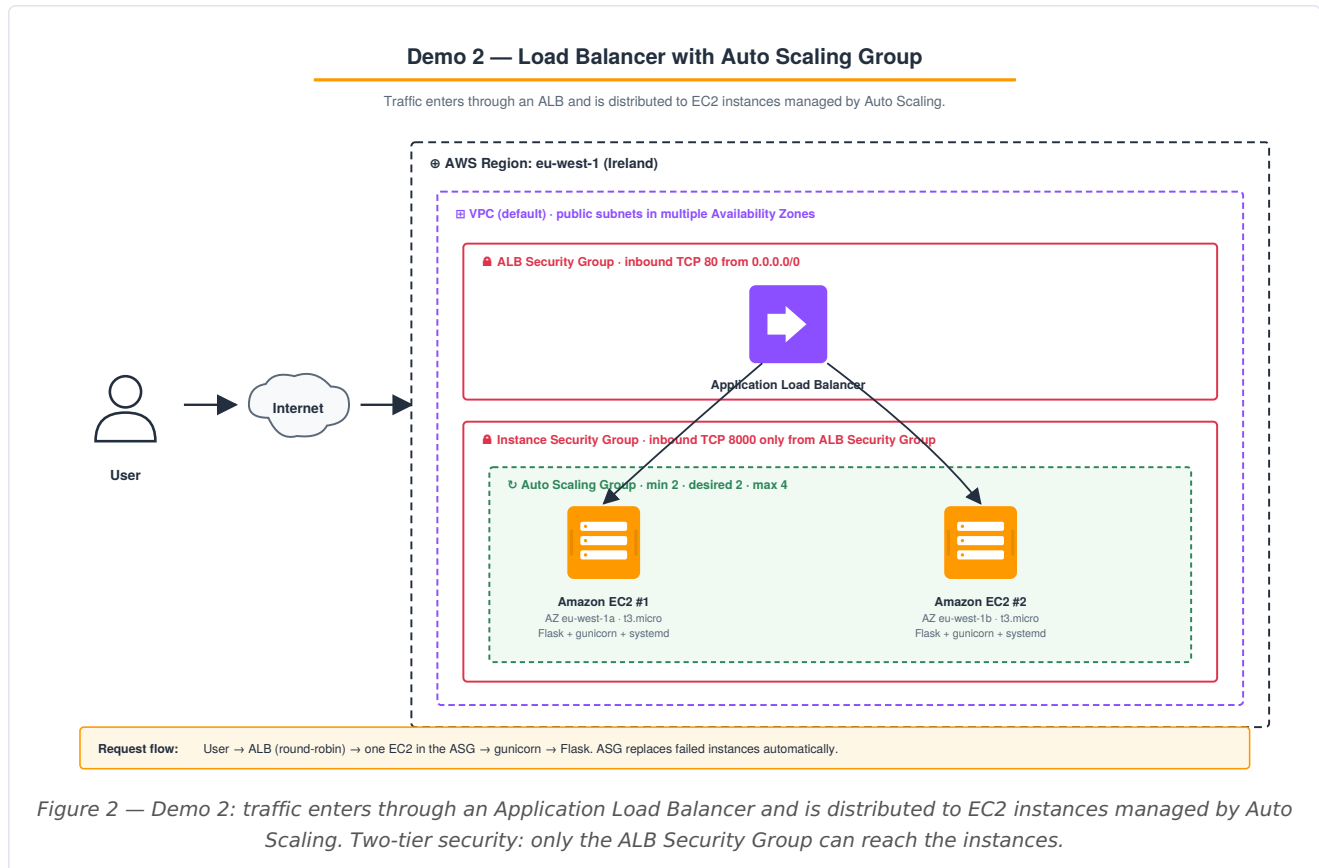
A Flask application running on one EC2 instance in `eu-west-1`. It is reachable from anywhere on the internet on port 8000. It auto-restarts if it crashes, and starts automatically if the machine reboots.

The architecture is simple — but it has one EC2 instance. If that instance fails, the app goes down. **Demo 2 fixes that** by adding a load balancer and multiple instances.

Do not terminate this instance yet. Demo 2 builds on top of it.

Demo 2 — Load Balancer with Auto Scaling Group

Goal: distribute traffic across multiple EC2 instances, replace failed instances automatically, and handle increased load by adding more instances.



What changes from Demo 1

The Flask application code does not change. What changes is the *infrastructure*:

- We add an **Application Load Balancer (ALB)** in front of the instances
- We replace the single instance with an **Auto Scaling Group (ASG)** that runs 2 to 4 instances
- We add a **second Security Group** so only the ALB can reach the instances (not the open internet)

The code is reused from Demo 1's GitHub repo.

Step 2.1 — Create the ALB Security Group

We need a separate Security Group for the ALB, distinct from the one for the instances.

In the AWS Console:

1. **EC2 -> Security Groups -> Create security group**

2. Name: `flask-alb-sg`

3. Description: `Security group for the ALB`

4. VPC: default

Inbound rule: Type `HTTP`, Source `Anywhere-IPv4`

Tags: `Project = python-chapter-demo`

Create security group.

Step 2.2 — Update the instance Security Group

We change `flask-demo-sg` so that only the ALB can reach the instances on port 8000.

In the AWS Console:

1. **Security Groups -> select `flask-demo-sg` -> Inbound rules tab -> Edit inbound rules**

2. Find the rule for port `8000` from `0.0.0.0/0` and click **Delete** on its row

3. Click **Add rule**:

- Type: `Custom TCP`

- Port range: `8000`

- Source: **Custom**, then in the search box select `flask-alb-sg`

- Description: `From ALB only`

4. Click **Save rules**

The SSH rule stays — we can still SSH in from our laptop.

Why this matters: the instances are no longer reachable from the internet directly. The only way in is through the ALB on port 8000. This is the **two-tier security pattern** — public-facing edge + protected internal tier.

Step 2.3 — Create a Target Group

A **Target Group** is the bridge between the ALB and the instances. The ALB sends traffic to the Target Group, and the Target Group keeps track of which instances are healthy.

Important rule: the **Target Group port must match the port the application listens on**. Our Flask app listens on port `8000`, so the Target Group is port `8000`. If they do not match, every health check fails and the ALB will not send traffic to any instance.

Here is the end-to-end path a request takes once the ALB is in front:

```
Browser --HTTP 80--> ALB Listener --> Target Group --HTTP 8000--> EC2 (gunicorn)
                    (public face)    (port 8000)                (Flask app)
```

The ALB **listens publicly on port 80** (so users hit a clean URL like `http://<alb-dns>/` with no port number),

and forwards each request to the Target Group on port **8000** — the port our gunicorn process is bound to. This split is intentional: it lets you keep the public URL clean while running the app on any internal port you like.

In the AWS Console:

1. **EC2** -> in the left sidebar under **Load Balancing**, click **Target Groups**
2. **Create target group**
3. Choose target type: **Instances**
4. Name: `flask-demo-tg`
5. Protocol: `HTTP`, Port: `8000` *(the port our Flask app listens on)*
6. VPC: default
7. Protocol version: `HTTP1`
8. **Health checks:**
 - Protocol: `HTTP`
 - Path: `/health`
9. Advanced health check settings (expand):
 - Healthy threshold: `2`
 - Unhealthy threshold: `2`
 - Interval: `15` seconds
 - Timeout: `5` seconds
10. Tags: `Project = python-chapter-demo`
11. **Next** -> on the "Register targets" page, do not add any targets yet -> **Create target group**

What is a health check? Every 15 seconds, the ALB asks each instance: "is `/health` on port 8000 returning 200 OK?" If two consecutive checks fail, the instance is marked unhealthy and the ALB stops sending it traffic until it recovers. Because the health check goes through the Target Group, it uses the **same port 8000** — another reason the Target Group port must match the app port.

Step 2.4 — Create the Application Load Balancer

In the AWS Console:

1. **EC2** -> in the left sidebar under **Load Balancing**, click **Load Balancers**
2. **Create load balancer** -> select **Application Load Balancer** -> **Create**
3. Name: `flask-demo-alb`
4. Scheme: **Internet-facing**
5. IP address type: **IPv4**
6. **Network mapping:**
 - VPC: default
 - Mappings: tick **all** Availability Zones (usually 3 in eu-west-1)
7. **Security groups:** remove `default`, add `flask-alb-sg`
8. **Listeners and routing:**
 - Protocol: `HTTP`, Port: `80` (*public-facing port — what the browser hits*)
 - Default action: **Forward to** `flask-demo-tg` (*which forwards to port 8000 on the instances*)
9. Tags: `Project = python-chapter-demo`
10. **Create load balancer**

Two different ports — recap:

- **Listener port** `80` = the port the **ALB itself** accepts public traffic on. This is what goes in the URL (`http://<alb-dns>/`).
- **Target Group port** `8000` = the port the **application** listens on inside each EC2 instance. This is gunicorn's `--bind 0.0.0.0:8000`.

The ALB ties these together: it receives on 80, forwards on 8000.

Provisioning takes 2-4 minutes. While we wait, build the Launch Template.

Step 2.5 — Create a Launch Template

A **Launch Template** is a saved blueprint for new EC2 instances. The Auto Scaling Group uses it to know *how* to create instances when scaling up.

In the AWS Console:

1. **EC2** -> in the left sidebar under **Instances**, click **Launch Templates**
2. **Create launch template**
3. Name: `flask-demo-lt`
4. Template version description: `Flask on Ubuntu 22.04 LTS`
5. **Application and OS Images:** in the **Quick Start** tab select **Ubuntu**, then choose **Ubuntu Server 22.04 LTS (HVM), SSD Volume Type** (64-bit x86)
6. **Instance type:** `t3.micro`
7. **Key pair:** `flask-demo-key`
8. **Network settings:** leave subnet at "Don't include in launch template" (the ASG will choose)
9. **Security groups:** `flask-demo-sg`
10. **Resource tags:** click **Add tag**: Key `Project`, Value `python-chapter-demo`. **Resource types:** select `Instances` and `Volumes`.
11. Expand **Advanced details** -> scroll to **User data** at the bottom -> paste the **same user-data script from Demo 1 Step 1.6** (remember to change `<your-username>` to your GitHub username)
12. **Create launch template**

Step 2.6 — Create the Auto Scaling Group

In the AWS Console:

1. **EC2** -> in the left sidebar under **Auto Scaling**, click **Auto Scaling Groups**
2. **Create Auto Scaling group**
3. Name: `flask-demo-asg`
4. Launch template: `flask-demo-lt` -> **Next**
5. **Network:**
 - VPC: default
 - Availability Zones and subnets: tick **all** default subnets -> **Next**
6. **Load balancing:** select **Attach to an existing load balancer**
 - **Choose from your load balancer target groups:** `flask-demo-tg`
 - Health checks: tick **Turn on Elastic Load Balancing health checks**
 - Health check grace period: `120` seconds
 - **Next**
7. **Configure group size:**
 - Desired capacity: `2`
 - Minimum: `2`
 - Maximum: `4`
 - **Next**
8. Notifications: skip -> **Next**
9. Tags: add `Project = python-chapter-demo`, tick **Tag new instances** -> **Next**
10. Review -> **Create Auto Scaling group**

What just happened: the ASG immediately starts launching 2 instances using the Launch Template. Each instance runs the user-data script (installs Python, clones the repo, starts the Flask service). After about 2-3 minutes, both instances will be running and healthy.

Step 2.7 — Test the load balancer

Wait 3-4 minutes for the instances to launch and pass health checks. Then:

1. **EC2 -> Load Balancers** -> click `flask-demo-alb`
2. In the Details panel, copy the **DNS name** (looks like `flask-demo-alb-xxxxx.eu-west-1.elb.amazonaws.com`)
3. Open `http://<dns-name>/` in your browser

Test from your laptop:

```
ALB_DNS=<your-alb-dns-name>

# Make several requests – you should be hitting different instances
for i in {1..6}; do
  curl http://$ALB_DNS/
  echo
done
```

To **see** the load balancer in action, edit `app.py` locally so the response includes the hostname, push, then trigger a refresh — but that is Demo 3's job. For now: trust that the ALB is distributing requests.

Step 2.8 — Verify high availability

Let's prove the ASG replaces failed instances automatically:

In the AWS Console:

1. **EC2 -> Instances**
2. Select **one** of the ASG instances (named like `flask-demo-asg`)
3. **Instance state -> Terminate instance** -> confirm

Watch what happens over the next 3-4 minutes:

- The terminated instance disappears
- The ASG notices it has only 1 instance (desired is 2)
- A new instance launches automatically
- The new instance bootstraps, becomes healthy, and joins the Target Group

That is **self-healing infrastructure** — no human intervention needed.

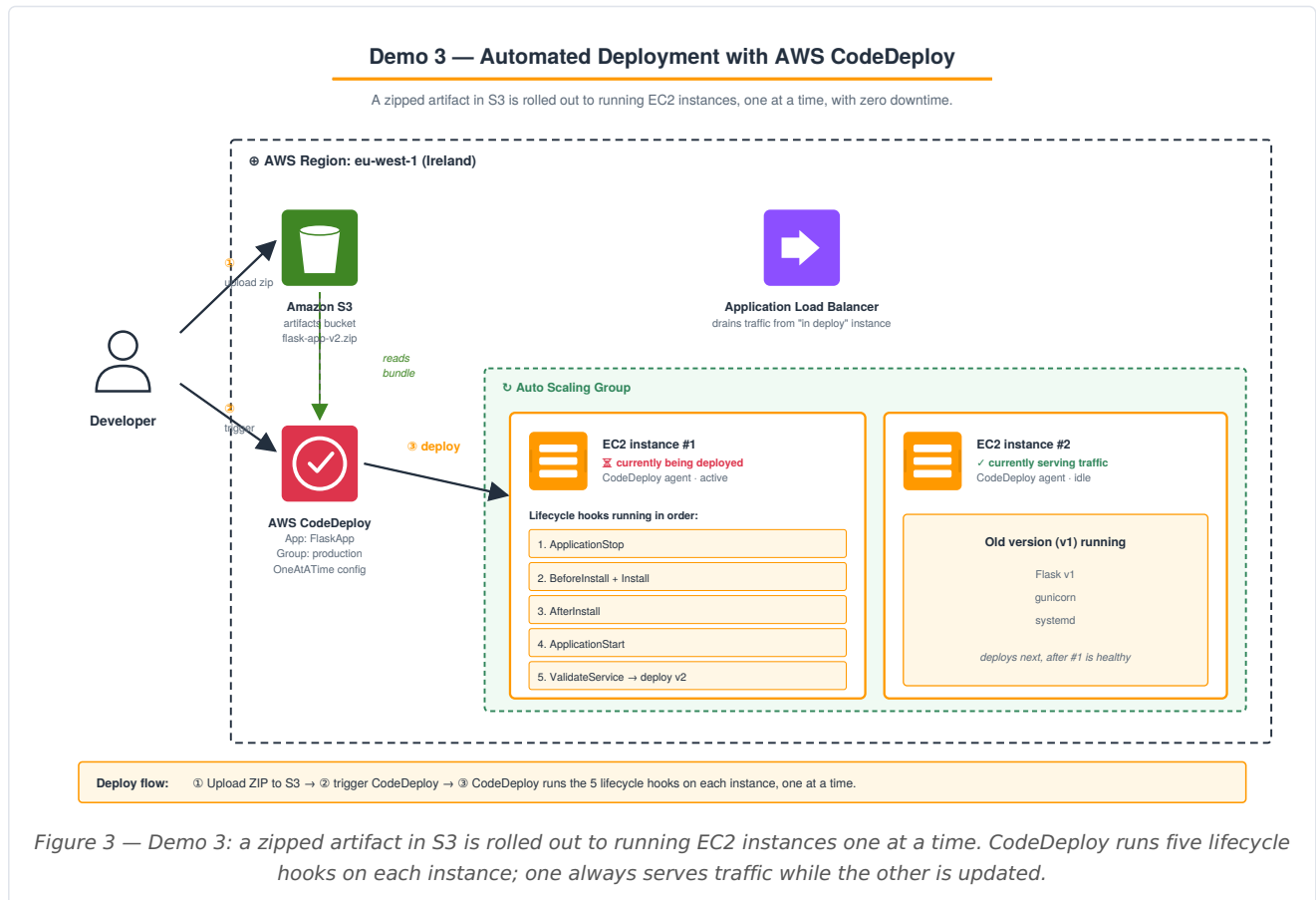
What we built in Demo 2

A load-balanced, self-healing Flask application across multiple AZs. If an instance fails, the ASG replaces it. If traffic doubles, we can change the ASG's desired capacity from 2 to 4 and have twice the capacity in 3 minutes.

Do not delete these resources yet. Demo 3 deploys new versions of the app to this same fleet.

Demo 3 — Automated Deployment with AWS CodeDeploy

Goal: deploy a new version of the Flask app to the ASG with one command — zero downtime, no SSH, no manual restart.



What changes from Demo 2

We extend the project with a deploy specification and lifecycle scripts:

```
flask-app/
├─ app.py
├─ requirements.txt
├─ templates/
│  └─ index.html
├─ appspec.yml
├─ deploy/
│  ├─ install.sh
│  ├─ start.sh
│  ├─ stop.sh
│  └─ validate.sh
└─ README.md

# NEW: tells CodeDeploy what to do
# NEW: lifecycle scripts
# - runs after files are unpacked
# - starts the service
# - stops the old version
# - checks the new version is healthy
```

We also need to make a few **infrastructure** changes:

- The EC2 instances need a small agent (the **CodeDeploy agent**) installed
- The instances need an **IAM role** so they can read from S3
- CodeDeploy itself needs an **IAM role** so it can talk to EC2

Step 3.1 — Add the deploy files locally

In your `flask-app` folder on your laptop, create the new files:

File: `appspec.yml`

```
version: 0.0
os: linux

files:
  - source: /
    destination: /home/ubuntu/flask-app

permissions:
  - object: /home/ubuntu/flask-app
    owner: ubuntu
    group: ubuntu

hooks:
  ApplicationStop:
    - location: deploy/stop.sh
      timeout: 60
      runas: root
  AfterInstall:
    - location: deploy/install.sh
      timeout: 300
      runas: root
  ApplicationStart:
    - location: deploy/start.sh
      timeout: 60
      runas: root
  ValidateService:
    - location: deploy/validate.sh
      timeout: 60
      runas: root
```

What is `appspec.yml`? It is CodeDeploy's recipe. It says "unpack the files to this folder, then run these scripts in this order". Without it, CodeDeploy does not know what to do with our zip.

Create the `deploy/` folder and the scripts:

```
mkdir -p deploy
```

File: `deploy/stop.sh`

```
#!/bin/bash
# Stop the running Flask service if it exists
if systemctl is-active --quiet flaskapp; then
    systemctl stop flaskapp
fi
exit 0
```

File: `deploy/install.sh`

```
#!/bin/bash
# CodeDeploy AfterInstall hook: prepare the venv and refresh the systemd unit.
set -eux

# Make sure python3-venv is present (CodeDeploy may run on a fresh AMI).
apt update -y
apt install -y python3 python3-venv python3-pip

cd /home/ubuntu/flask-app

# Create the venv if it does not exist, then (re)install dependencies.
[ -d venv ] || python3 -m venv venv
./venv/bin/pip install -r requirements.txt

chown -R ubuntu:ubuntu /home/ubuntu/flask-app

# Refresh the systemd unit in case the ExecStart or env has changed.
cat > /etc/systemd/system/flaskapp.service <<'EOF'
[Unit]
Description=Flask App
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/flask-app
ExecStart=/home/ubuntu/flask-app/venv/bin/gunicorn --workers 3 --bind 0.0.0.0:8000 app:app
Restart=always

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
```

File: `deploy/start.sh`

```
#!/bin/bash
systemctl enable flaskapp
systemctl start flaskapp
```

File: `deploy/validate.sh`

```
#!/bin/bash
# Wait up to 30 seconds for the app to respond on /health
for i in {1..30}; do
    if curl -fs http://localhost/health > /dev/null; then
        echo "Service is healthy"
        exit 0
    fi
    sleep 1
done

echo "Service did not become healthy in time"
exit 1
```

Make all scripts executable and push to GitHub:

```
chmod +x deploy/*.sh

git add appspec.yml deploy/
git commit -m "Add CodeDeploy lifecycle scripts"
git push origin main
```

Step 3.2 — Bump the version so we can see the deployment

We want to [see](#) the rolling deploy happen in the browser. The cleanest way is to change one line — `APP_VERSION` — so the orange badge on the page changes from `v1` to `v2` as each instance is updated.

Open `app.py` and change just this line:

```
APP_VERSION = "v1"
```

to:

```
APP_VERSION = "v2"
```

That is the only change. The styled HTML page, the `/echo` JSON, and the `/health` endpoint all stay the same — they just pick up the new version string.

Test locally one more time:

```
source venv/bin/activate
python app.py
```

Open `http://localhost:8000/` in the browser — the orange badge should now read `v2`. Stop with `Ctrl+C`.

Commit but **do not push yet** — we will deploy this through CodeDeploy.

```
git add app.py
git commit -m "Bump app version to v2"
```

Step 3.3 — Create an S3 bucket for deployment artifacts

CodeDeploy reads our application bundle from S3.

In the AWS Console:

1. Search for **S3** in the top search bar and open it
2. **Create bucket**
3. Bucket name: `flask-demo-artifacts-<your-initials>-<random-number>` (must be globally unique). The author of this guide uses `flask-demo-artifacts-cnm-9472` — pick something similar with **your** initials and a random number.
4. AWS Region: **eu-west-1**
5. Object Ownership: **ACLs disabled** (default)
6. Block all public access: **leave ticked**
7. Bucket Versioning: **Enable** (lets us roll back to old versions)
8. Tags: `Project = python-chapter-demo`
9. **Create bucket**

Remember this bucket name — we will use it in the CodeDeploy step.

Important — write down your exact bucket name now. You will use it in three places later:

1. **Step 3.9** when triggering the deployment in the Console.
2. **Section A — local** `.env` as the value of `S3_BUCKET`.
3. **Section A — GitHub Actions Secrets** as the value of the `S3_BUCKET` secret (so the pipeline knows which bucket to upload to).

All three must use the **exact same name** — a typo in any one of them breaks the deployment. If you ever change the bucket later, update both your `.env` and the `S3_BUCKET` secret in GitHub.

Step 3.4 — Create the IAM role for EC2 instances

The instances need permission to read from our S3 bucket.

In the AWS Console:

1. Search for **IAM** -> open it
2. Left sidebar -> **Roles** -> **Create role**
3. Trusted entity type: **AWS service**
4. Use case: **EC2** -> **Next**
5. **Permissions:** in the search box type and select these two policies (tick each one):
 - `AmazonSSMManagedInstanceCore`
 - `AmazonS3ReadOnlyAccess`
6. **Next**
7. Role name: `EC2CodeDeployRole`
8. Tags: `Project = python-chapter-demo`
9. **Create role**

Why these two policies? `AmazonSSMManagedInstanceCore` lets the CodeDeploy agent talk to AWS.
`AmazonS3ReadOnlyAccess` lets the instance download our zip from S3.

Now attach this role to the Launch Template (so all new instances get it):

1. **EC2 -> Launch Templates** -> select `flask-demo-lt` -> **Actions -> Modify template (Create new version)**
2. Scroll to **Advanced details** -> **IAM instance profile** -> select `EC2CodeDeployRole`
3. Scroll to **User data** and **add these lines at the END of the existing script**:

```
# Install the CodeDeploy agent (Ubuntu)
apt install -y ruby-full wget
cd /tmp
wget https://aws-codedeploy-eu-west-1.s3.eu-west-1.amazonaws.com/latest/install
chmod +x ./install
./install auto > /tmp/codedeploy-install.log 2>&1
systemctl enable codedeploy-agent
systemctl start codedeploy-agent
```

4. **Create template version**

Now make this new version the default:

1. Still on `flask-demo-lt`, **Actions -> Set default version** -> pick the new version number -> **Set as default**

Step 3.5 — Refresh the ASG instances so they get the agent

The existing instances do not have the CodeDeploy agent or the IAM role. We replace them.

1. **EC2 -> Auto Scaling Groups** -> select `flask-demo-asg`
2. **Instance refresh** tab -> **Start instance refresh**
3. Minimum healthy percentage: `50%`
4. **Start instance refresh**

This launches new instances (with the agent + IAM role), waits for each to be healthy, then terminates the old ones. Takes about 5-7 minutes.

Why instance refresh? It replaces instances gradually so the app stays up the whole time.

Step 3.6 — Create the CodeDeploy IAM role

CodeDeploy itself needs permission to talk to EC2.

In the AWS Console:

1. **IAM -> Roles -> Create role**
2. Trusted entity: **AWS service**
3. Use case: from the dropdown select **CodeDeploy**, choose **CodeDeploy** (the basic one, not for Lambda/ECS) -> **Next**
4. **Permissions:** the policy `AWSCodeDeployRole` is auto-attached. Click **Next**.
5. Role name: `CodeDeployServiceRole`
6. Tags: `Project = python-chapter-demo`
7. **Create role**

Step 3.7 — Create the CodeDeploy application and deployment group

In the AWS Console:

1. Search for **CodeDeploy** -> open it
2. Left sidebar -> **Applications -> Create application**
3. Application name: `flask-demo-app`
4. Compute platform: **EC2/On-premises**
5. Tags: `Project = python-chapter-demo`
6. **Create application**

Now create a Deployment Group inside the application:

1. On the application page, scroll to **Deployment groups -> Create deployment group**
2. Deployment group name: `flask-demo-dg`
3. **Service role:** `CodeDeployServiceRole`
4. **Deployment type:** **In-place**
5. **Environment configuration:** tick **Amazon EC2 Auto Scaling groups** -> select `flask-demo-asg`
6. **Deployment configuration:** `CodeDeployDefault.OneAtATime`
7. **Load balancer:** tick **Enable load balancing** -> **Application Load Balancer or Network Load Balancer** -> select `flask-demo-tg`
8. Tags: `Project = python-chapter-demo`
9. **Create deployment group**

What is `OneAtATime`? CodeDeploy will deploy to one instance at a time. While that instance is updating, the ALB drains its traffic and sends 100% to the others. Once it is healthy, CodeDeploy moves to the next one. **Zero downtime.**

Step 3.8 — Package and upload the app bundle

Back on your laptop, in the `flask-app` folder:

```
# Make sure we have the latest local changes committed (not pushed)
git status # Should show "nothing to commit, working tree clean"

# Create the zip
zip -r flask-app-v2.zip \
    app.py requirements.txt appspec.yml deploy/ templates/

# Verify the contents
unzip -l flask-app-v2.zip
```

Upload to S3:

In the AWS Console:

1. **S3** -> open your `flask-demo-artifacts-cnm-9472` bucket (or whichever name you chose in Step 3.3)
2. **Upload** -> **Add files** -> select `flask-app-v2.zip`
3. **Upload**

Step 3.9 — Trigger the deployment

In the AWS Console:

1. **CodeDeploy** -> **Applications** -> click `flask-demo-app`
2. Scroll to **Deployment groups**, select `flask-demo-dg`
3. Top right: **Create deployment**
4. Deployment group: `flask-demo-dg` (pre-filled)
5. Revision type: **My application is stored in Amazon S3**
6. Revision location: `s3://flask-demo-artifacts-cnm-9472/flask-app-v2.zip` (substitute your own bucket name)
7. Revision file type: `.zip`
8. Leave everything else default
9. **Create deployment**

You will land on a deployment progress page. Watch it for 3-5 minutes:

- Phase 1: deploying to instance #1 (the other one keeps serving traffic)
- Phase 2: deploying to instance #2

Meanwhile, in another browser tab, open `http://<alb-dns-name>/` and refresh repeatedly. You will see:

- Some refreshes still show the `v1` badge (the old version, served by instance #2)
- Some refreshes start showing the `v2` badge (after deployment to instance #1)
- Eventually every refresh shows `v2` (once both instances are updated)

The `Served by` line on the page changes too — you can literally watch the load balancer alternating between instances. **Zero downtime — the app stayed up the whole time.**

What we built in Demo 3

Repeatable, automated, zero-downtime deployment. To deploy a new version:

1. Edit code locally, commit, test
2. `zip -r flask-app-vN.zip ...`
3. Upload to S3
4. Trigger CodeDeploy

That is the deploy pipeline. **Section A** (optional) shows how to put this whole thing on autopilot using GitHub Actions.

Cleanup — Delete everything we created

Run these in order to avoid AWS bills.

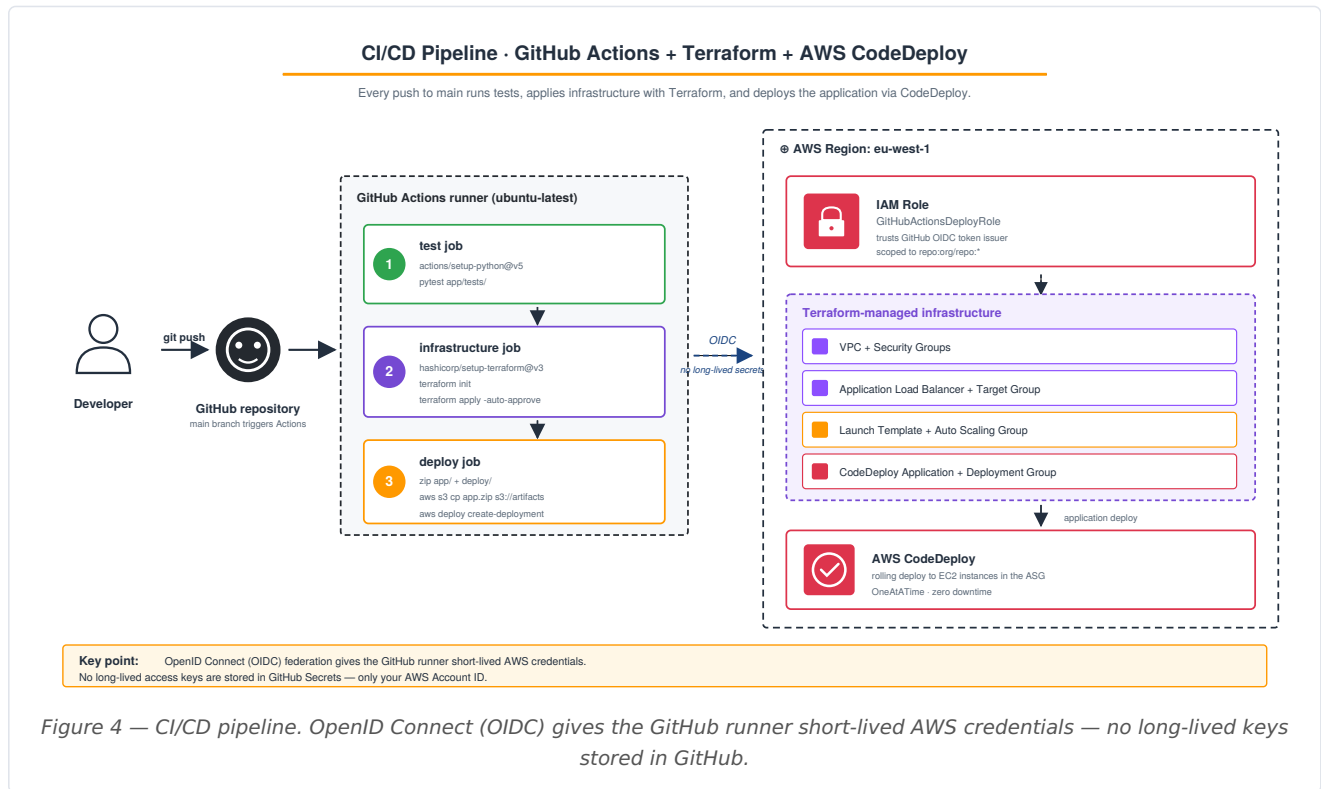
In the AWS Console:

1. **CodeDeploy** -> **Applications** -> select `flask-demo-app` -> **Delete application**
2. **EC2** -> **Auto Scaling Groups** -> select `flask-demo-asg` -> **Delete** -> type "delete" to confirm
3. **EC2** -> **Launch Templates** -> select `flask-demo-lt` -> **Actions** -> **Delete template**
4. **EC2** -> **Load Balancers** -> select `flask-demo-alb` -> **Actions** -> **Delete**
5. **EC2** -> **Target Groups** -> select `flask-demo-tg` -> **Actions** -> **Delete**
6. **EC2** -> **Instances** -> if any remain, select -> **Instance state** -> **Terminate**
7. **EC2** -> **Security Groups** -> delete `flask-alb-sg` and `flask-demo-sg`
8. **EC2** -> **Key Pairs** -> delete `flask-demo-key`
9. **S3** -> open `flask-demo-artifacts-cnm-9472` (or whichever bucket name you used) -> **Empty** -> then **Delete**
10. **IAM** -> **Roles** -> delete `EC2CodeDeployRole` and `CodeDeployServiceRole`

Tip for next time: use **Tag-based search** to find every resource tagged `Project = python-chapter-demo` and delete in bulk via the **Resource Groups & Tag Editor** service.

Section A — CI/CD with GitHub Actions (optional, ~20 minutes)

Goal: every push to `main` automatically builds, tests, packages, uploads to S3, and triggers a CodeDeploy deployment.



What we add

```
flask-app/
├── ... (existing files)
├── .env.example      # NEW: template showing required variables
├── .gitignore        # UPDATED: ignore .env so secrets never leak
├── .github/
│   └── workflows/
│       └── deploy.yml # NEW: the CI/CD workflow
```

Important — secrets management rule: sensitive values (role ARNs, account IDs in URLs, API keys) must **never** be hardcoded in tracked files. They go in two places:

- **Local development:** in a `.env` file on your laptop. This file is `.gitignored` and never reaches GitHub.
- **CI/CD pipeline:** in **GitHub Actions** at the repository level. GitHub gives you two separate stores:
 - **Variables** (`vars.*`): non-sensitive config like Region, bucket name, app name. Collaborators can read these.
 - **Secrets** (`secrets.*`): sensitive values like the role ARN. Even repo collaborators with write access cannot read these back — only overwrite them.

The `.env.example` template lists everything the project needs; the real values get filled into both your local `.env` and the right GitHub store (vars or secrets) based on how sensitive each one is.

Step A.1 — Create the GitHub Actions IAM role

In the AWS Console:

1. **IAM -> Identity providers -> Add provider**
2. Provider type: **OpenID Connect**
3. Provider URL: `https://token.actions.githubusercontent.com`
4. Audience: `sts.amazonaws.com`
5. **Add provider**

Now create the role:

1. **IAM -> Roles -> Create role**
2. Trusted entity: **Web identity**
3. Identity provider: `token.actions.githubusercontent.com`
4. Audience: `sts.amazonaws.com`
5. GitHub organization: `<your-username>`
6. GitHub repository: `flask-app`
7. Branch: `main`
8. **Next**
9. Permissions: attach
 - `AmazonS3FullAccess` (scope this down in production)
 - `AWSCodeDeployDeployerAccess`
10. Role name: `GitHubActionsDeployRole`
11. **Create role**

Copy the role's **ARN** (looks like `arn:aws:iam::123456789012:role/GitHubActionsDeployRole`) — you need it in the next step.

Step A.2 — Create `.env.example` and your local `.env`

The `.env.example` file is the **single source of truth** for what configuration the project needs. It is committed to

the repo so anyone (including your future self) can see what variables to set.

File: `.env.example`

```
# --- Sensitive (will become a GitHub Secret) ---

# Full ARN of the IAM role GitHub Actions assumes via OIDC.
# Format: arn:aws:iam::<account-id>:role/<role-name>
# The account ID makes this sensitive – keep this one as a secret.
AWS_ROLE_ARN=arn:aws:iam::123456789012:role/GitHubActionsDeployRole

# --- Non-sensitive config (will become GitHub Variables) ---

# AWS region where all resources live
AWS_REGION=eu-west-1

# S3 bucket where deployment artifacts are uploaded
S3_BUCKET=flask-demo-artifacts-cnm-9472

# CodeDeploy application + deployment group from Demo 3
CODEDEPLOY_APP=flask-demo-app
CODEDEPLOY_GROUP=flask-demo-dg
```

Now copy it to a real `.env` file with **your actual values** filled in:

```
cp .env.example .env

# Edit .env with your real values
nano .env
# Replace AWS_ROLE_ARN with your full role ARN (account ID is the 12-digit
# number in the top-right of the AWS Console)
# Replace S3_BUCKET with the bucket suffix you used in Demo 3
```

Update `.gitignore` so `.env` is never committed:

```
# Append .env to the existing .gitignore
echo ".env" >> .gitignore
```

Why this pattern? The `.example` file is your documentation — it tells contributors what variables exist. The real `.env` is your private config — it has the actual values. Committing `.env.example` and ignoring `.env` is the standard convention used in nearly every modern project.

You can now use these locally for one-off manual deployments. Source the file and the values become available to the shell:

```
# Load .env into the current shell (so $AWS_REGION etc. work in commands)
set -a; source .env; set +a

# Verify
echo "Region: $AWS_REGION"
echo "Bucket: $S3_BUCKET"

# Example: manually upload an artifact using these values
aws s3 cp flask-app-v2.zip \
  "s3://$S3_BUCKET/flask-app-v2.zip" \
  --region "$AWS_REGION"
```


Step A.3 — Add the same values to GitHub (Variables + Secrets)

The CI/CD workflow runs on GitHub's servers, so it cannot read your local `.env`. We mirror the same values into two separate GitHub stores: **Variables** for non-sensitive config, **Secrets** for the sensitive role ARN.

On GitHub (in your browser):

1. Open your repository: `https://github.com/<your-username>/flask-app`
2. Click **Settings** (top tab, far right)
3. In the left sidebar, expand **Secrets and variables** -> click **Actions**

You will see two tabs at the top of the page: **Secrets** and **Variables**. Use both.

Add four Variables (click the **Variables** tab, then **New repository variable**):

Variable name	Value
<code>AWS_REGION</code>	<code>eu-west-1</code>
<code>S3_BUCKET</code>	your S3 bucket name (e.g. <code>flask-demo-artifacts-cnm-9472</code>)
<code>CODEDEPLOY_APP</code>	<code>flask-demo-app</code>
<code>CODEDEPLOY_GROUP</code>	<code>flask-demo-dg</code>

Add one Secret (click the **Secrets** tab, then **New repository secret**):

Secret name	Value
<code>AWS_ROLE_ARN</code>	full ARN of the role you created in Step A.1, e.g. <code>arn:aws:iam::123456789012:role/GitHubActionsDeployRole</code>

Why split them up? The role ARN contains your **AWS Account ID** — a piece of information AWS treats as semi-sensitive. Putting it in **Secrets** means even collaborators with write access cannot read it back. The region, bucket name, and CodeDeploy names are not sensitive — putting them in **Variables** lets your team see how the pipeline is configured at a glance.

About visibility: GitHub stores Secrets encrypted; the value is only decrypted inside the workflow runner at runtime. Variables are visible to anyone who can view the repository's settings — never put credentials, ARNs that contain account IDs, or API keys in Variables.

Step A.4 — Add the workflow file

```
mkdir -p .github/workflows
```

File: `.github/workflows/deploy.yml`

```

name: Deploy to AWS

on:
  push:
    branches: [main]

# OIDC token is needed so this workflow can assume the AWS IAM role below.
permissions:
  id-token: write
  contents: read

# Pipeline config – set these in GitHub repo settings:
# Secrets: AWS_ROLE_ARN
# Variables: AWS_REGION, S3_BUCKET, CODEDEPLOY_APP, CODEDEPLOY_GROUP
env:
  AWS_REGION: ${ vars.AWS_REGION }
  S3_BUCKET: ${ vars.S3_BUCKET }
  CODEDEPLOY_APP: ${ vars.CODEDEPLOY_APP }
  CODEDEPLOY_GROUP: ${ vars.CODEDEPLOY_GROUP }

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies (for tests)
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest

      - name: Run tests
        # Replace with real tests; for the demo we just verify the module imports.
        run: |
          python -c "import app; print('App imports OK')"

      - name: Configure AWS credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.AWS_ROLE_ARN }
          aws-region: ${ env.AWS_REGION }

      - name: Package application
        run: |
          zip -r app-${ github.sha }.zip \
            app.py requirements.txt appspec.yml deploy/ templates/

      - name: Upload bundle to S3
        run: |
          aws s3 cp app-${ github.sha }.zip \
            s3://${ env.S3_BUCKET }/app-${ github.sha }.zip

      - name: Trigger CodeDeploy deployment
        run: |
          aws deploy create-deployment \

```

```
--application-name ${ env.CODEDEPLOY_APP } \
--deployment-group-name ${ env.CODEDEPLOY_GROUP } \
--s3-location bucket=${ env.S3_BUCKET },key=app-${ github.sha }.zip,bundleType=zip
```

Notice: the workflow uses **both** stores — `${{ vars.XXX }}` for non-sensitive config and `${{ secrets.AWS_ROLE_ARN }}` for the role ARN. The `env:` block at the top maps `vars.*` into shorter `env.*` references so the rest of the steps stay readable.

Tip — promoting `vars` to top-level `env`: the workflow does `env: { AWS_REGION: ${{ vars.AWS_REGION }}` once at the top, then every step refers to `${{ env.AWS_REGION }}`. This way, if you ever rename a variable in GitHub settings, you only update one place in the YAML.

Step A.5 — Push and watch it deploy

```
git add .env.example .gitignore .github/
git commit -m "Add CI/CD workflow with secret-driven config"
git push origin main
```

Verify before pushing: run `git status` and confirm `.env` does **not** appear in the list of tracked files. If it does, your `.gitignore` is wrong and your account ID is about to be public. Stop and fix it before pushing.

On GitHub, open your repository -> **Actions** tab. You should see the workflow running. After 3-5 minutes the deployment finishes and your ALB serves the new version.

From now on, **every `git push` to main is a deployment**. No manual zip, no Console clicks. To rotate any value (move buckets, change region, switch role), update the corresponding GitHub Secret — no code change required.

Step A.6 — Troubleshooting common failures

If a workflow run fails, check the **Actions** tab on GitHub and click the failed step. The most common issues:

- **Could not load credentials from any providers** — the `AWS_ACCOUNT_ID` or `AWS_ROLE_NAME` secret is wrong, or the IAM trust policy on `GitHubActionsDeployRole` does not allow the repo/branch.
- **Access Denied (S3)** — the role's policies are too narrow. Re-attach `AmazonS3FullAccess` for the demo (scope it down in production).
- **No such bucket** — `S3_BUCKET` secret has a typo, or you are in the wrong region.
- **CodeDeploy fails on instances** — open the deployment in the CodeDeploy console and click the failing instance to see the per-hook log; usually `install.sh` or `validate.sh` has the answer.